UNIT - V
Managing and Maintaining the Code:

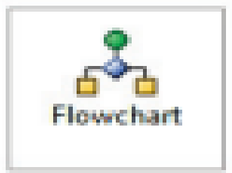
Project organization

- While working on any automation project, it is very important to work with a proper set of rules so that the project can be organized in an efficient way.
- In UiPath, the following are some of the best practices considered while working on a project:
- Pick an appropriate layout for each workflow Break the whole process into smaller parts
- Use exception handling Make your workflow readable Keep it clean

- Picking an appropriate layout for each workflow
- There are various layouts available while creating a new project.
- Among those layouts, we have to choose the best option on the basis of the type of automation process we are undertaking.
- All the layouts are shown in the following screenshot

Blank

An empty project. A diagram is a graphical representation of a business process.

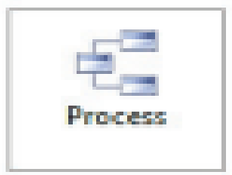


Simple Process

Model a process as a flowchart diagram.

Agent Process Improvement

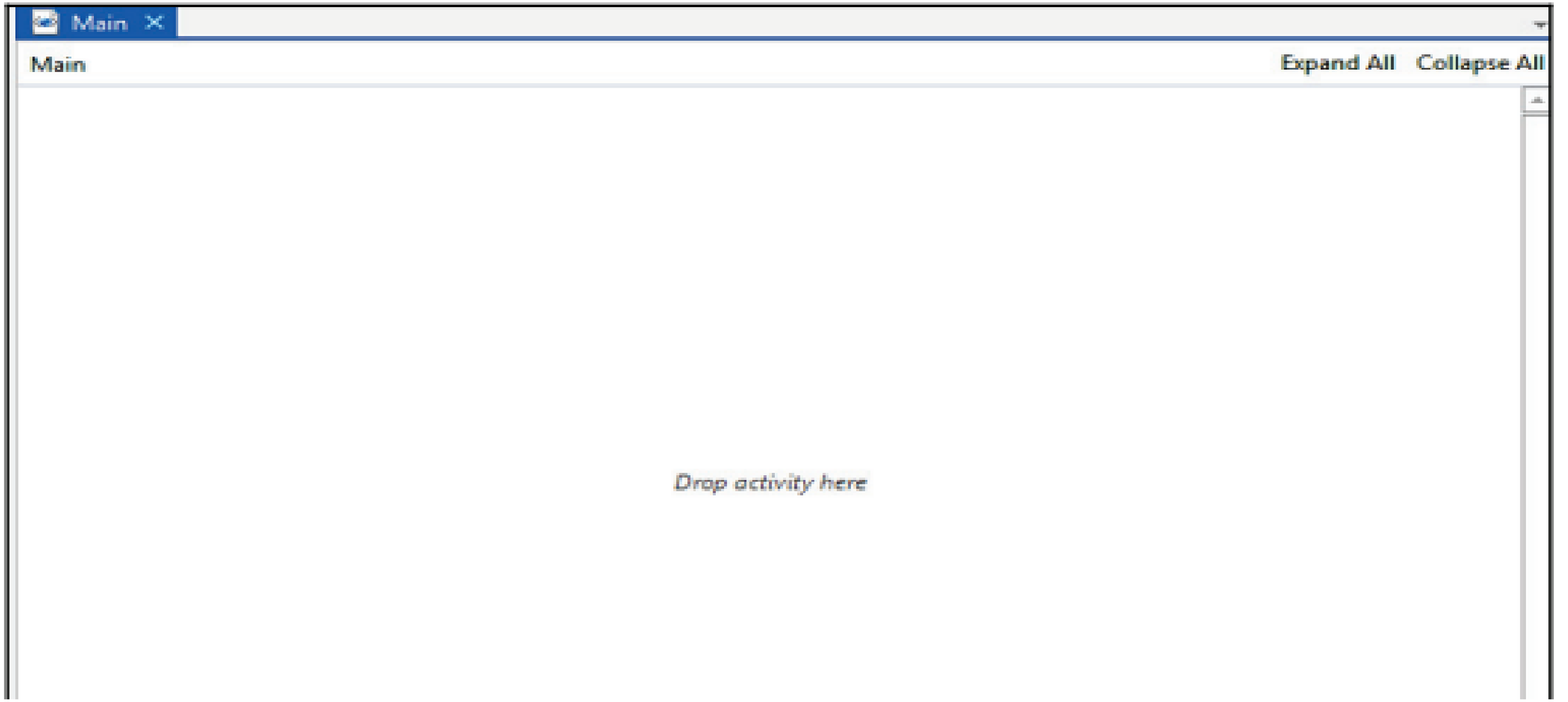
Trigger an automation in response to a mouse or keyboard user event.



Transactional Business Process

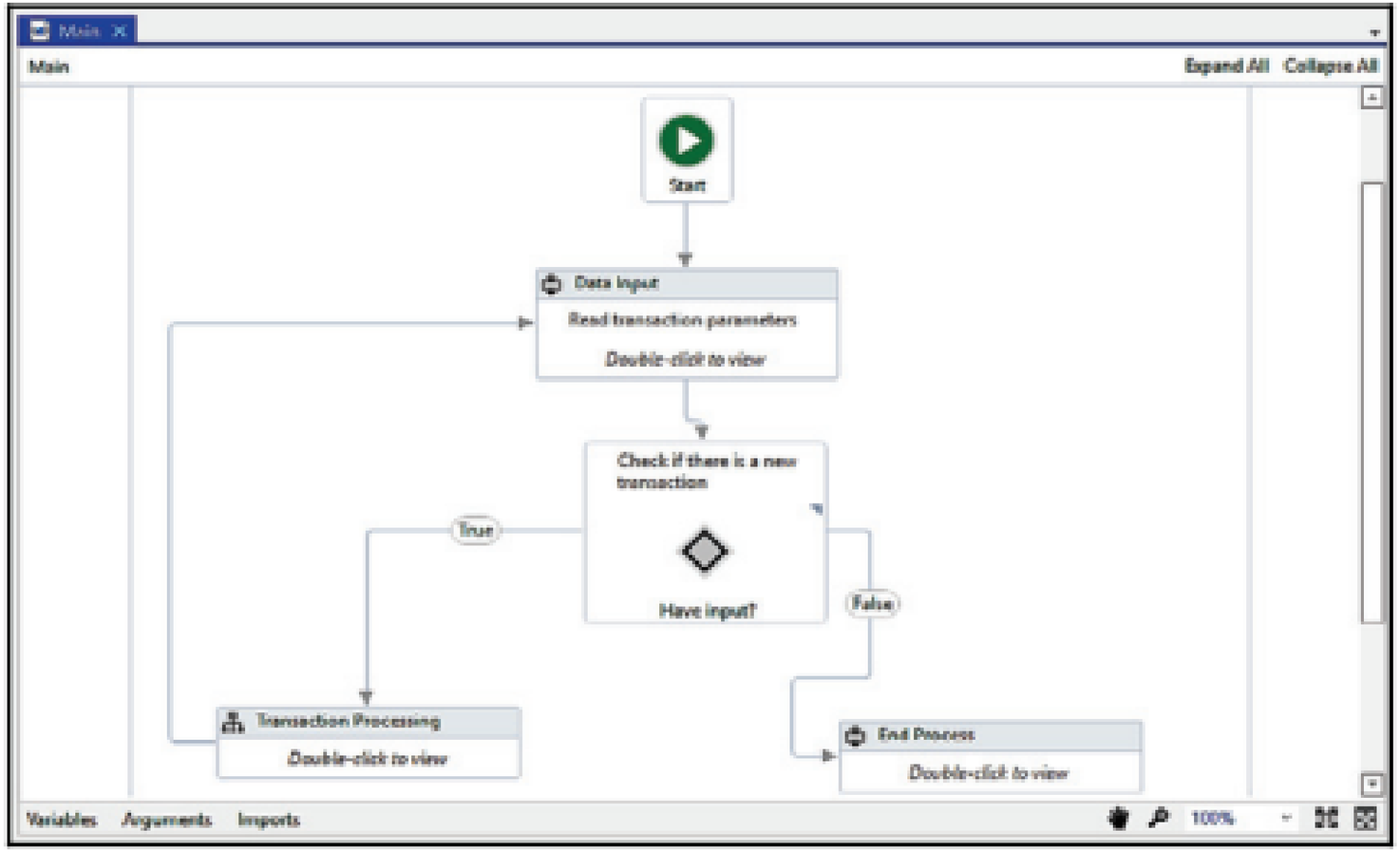
Model a business process as a state machine diagram.

- Blank A Blank project is simply a blank page on which you can create the type of layout you want. That is, you can simply start with a Sequence activity if your workflow is in a single order/sequence or you can use a Flowchart activity if you have a bigger or more complex workflow to be designed. It depends on the needs of the user or the type of automation to be undertaken

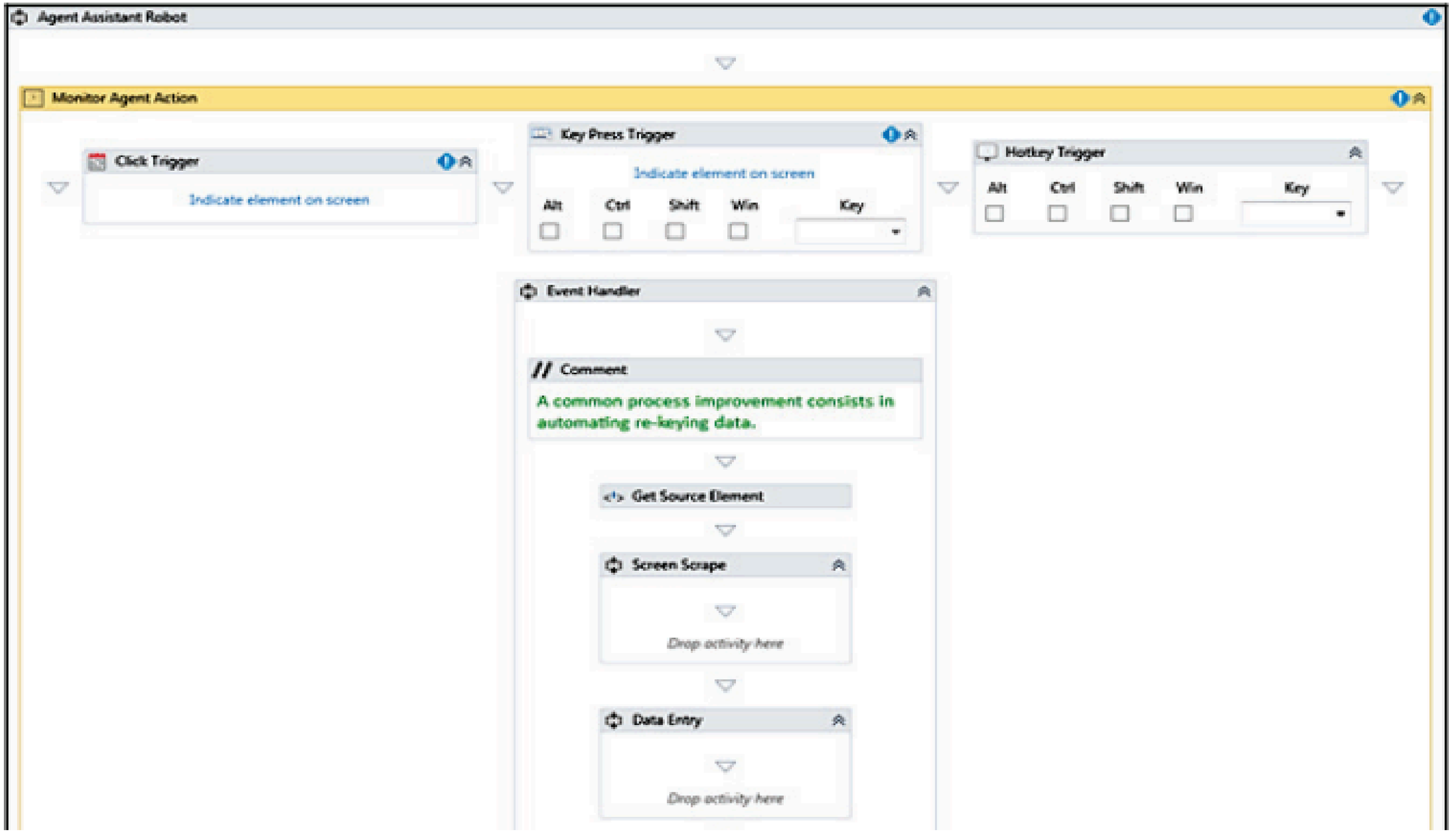

Main
Main
Expand All
Collapse All
Drop activity here

Simple process

- The simple process is a layout that is used to model a process as a flowchart diagram in which there is space for user input. Inside this, we can use a sequence that processes the required input in a further transaction process. If there is no new input for the transaction, it will end the process; inside the transaction process, we have to make a workflow that can be used to automate it. This is by default a generated process that can be deleted or changed if required


Main
Main
Expand All
Collapse All
Start
Data Input
Read transaction parameters
Double-click to view
Check if there is a new transaction
True
Have input?
False
Transaction Processing
Double-click to view
End Process
Double-click to view
Variables
Arguments
Imports
100%

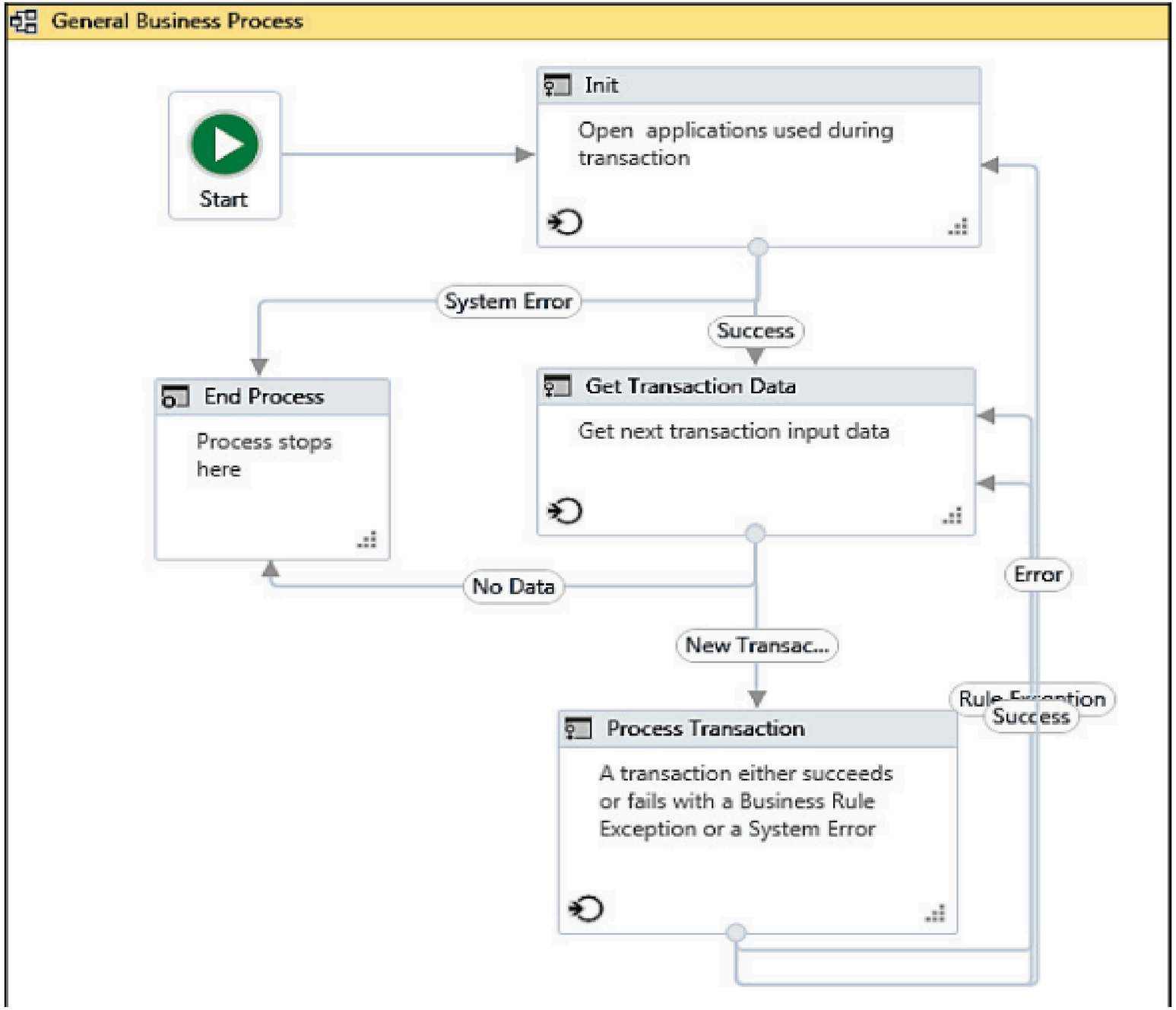
- Agent process improvement This triggers the automation in response to a mouse or keyboard user event. It is basically used when the user is automating processes that involve typing or clicking actions


Agent Assistant Robot
Monitor Agent Action
Click Trigger
Indicate element on screen
Key Press Trigger
Indicate element on screen
Alt
Ctrl
Shift
Win
Key
Hotkey Trigger
Alt
Ctrl
Shift
Win
Key
Event Handler
Comment
A common process improvement consists in automating re-keying data.
Get Source Element
Screen Scrape
Drop activity here
Data Entry
Drop activity here

- Transactional business process We will use this layout if we want to model a business process as a State Machine diagram. It is basically a demo of how transactional business process automation works. If we want to build a better Robot to automate such processes, it is better to use this layout. This layout is categorized into different states

- Init: In Init state, we have to configure our settings, credentials (if any), and initialize all the variables that are going to be used in this transaction. All configuration files of the applications (being used in this transaction), are read and taken into account by the robot. The Init state also invokes all the applications that are used in the transaction

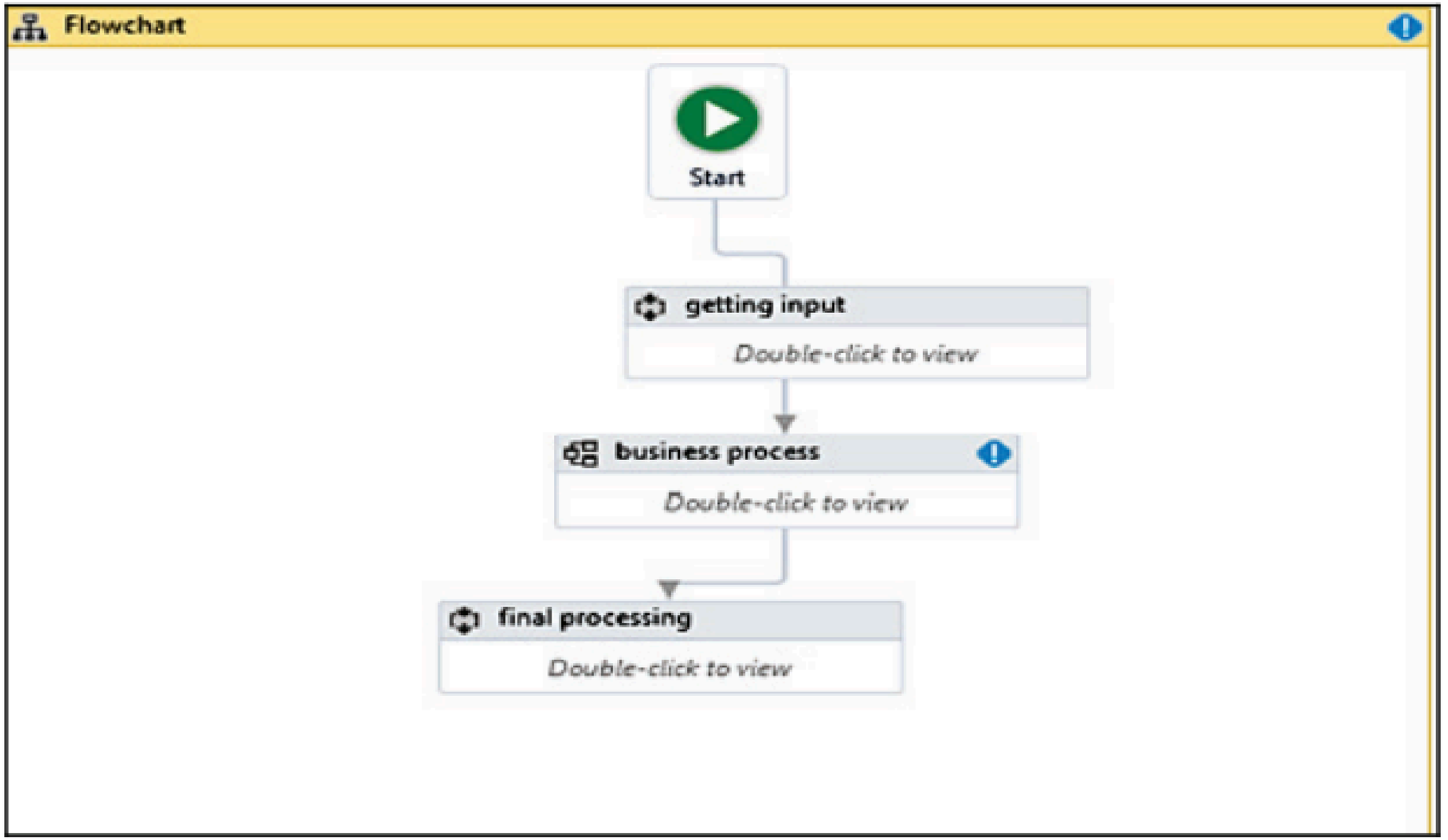
- Get Transaction Data: In this state, all the transaction data is fetched from the Init state. If there is no transaction data, then it transfers the control to End Process state. Process Transaction: In this State, all the transaction data is processed. End Process: This state ensures that all the processes are completed and there is no transaction data available. It also closes all the applications that are used in the transaction


General Business Process
Start
Init
Open applications used during transaction
System Error
Success
End Process
Process stops here
Get Transaction Data
Get next transaction input data
No Data
Error
New Transac...
Rule Exception
Success
Process Transaction
A transaction either succeeds or fails with a Business Rule Exception or a System Error

Breaking the process into smaller parts

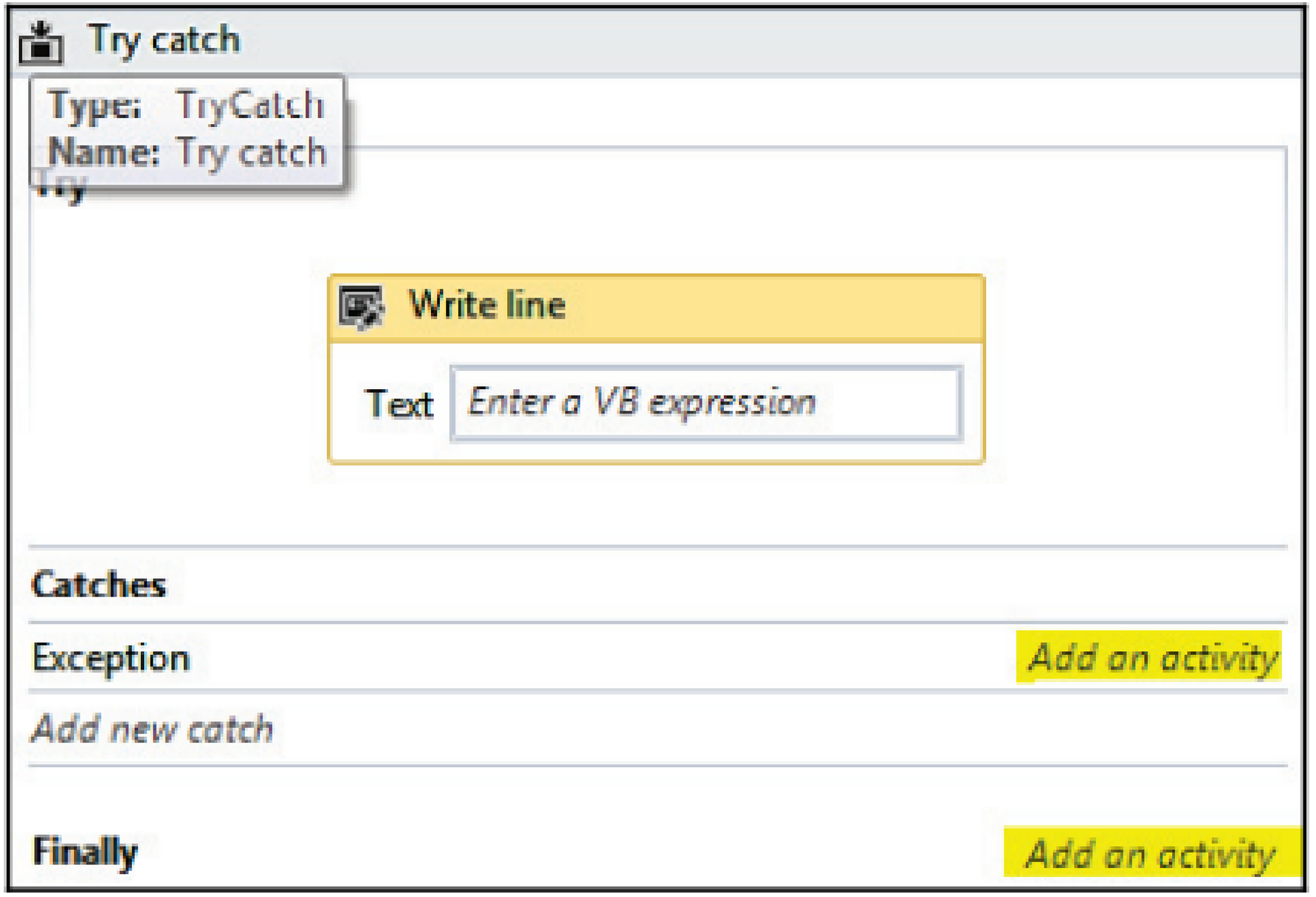
- To build any project, we have to use various activities.
  - But using too many activities makes the project clumsy and it is not readable.
- We have to design our project in such a way that each independent part resides alone.
- We can achieve this by using workflows.
- We should put each independent part of the project inside a single workflow.

- We can invoke all the workflows inside the project at the appropriate position.
- Dividing the project into workflows makes the project cleaner and more maintainable.
- Now, if any developer wants to debug your code, they can check the different workflows and easily pinpoint in which workflow a particular error occurred.
- If the project is not divided into workflows, it will be a nightmare for the developer to fix any error.


Flowchart
Start
getting input
Double-click to view
business process
Double-click to view
final processing
Double-click to view

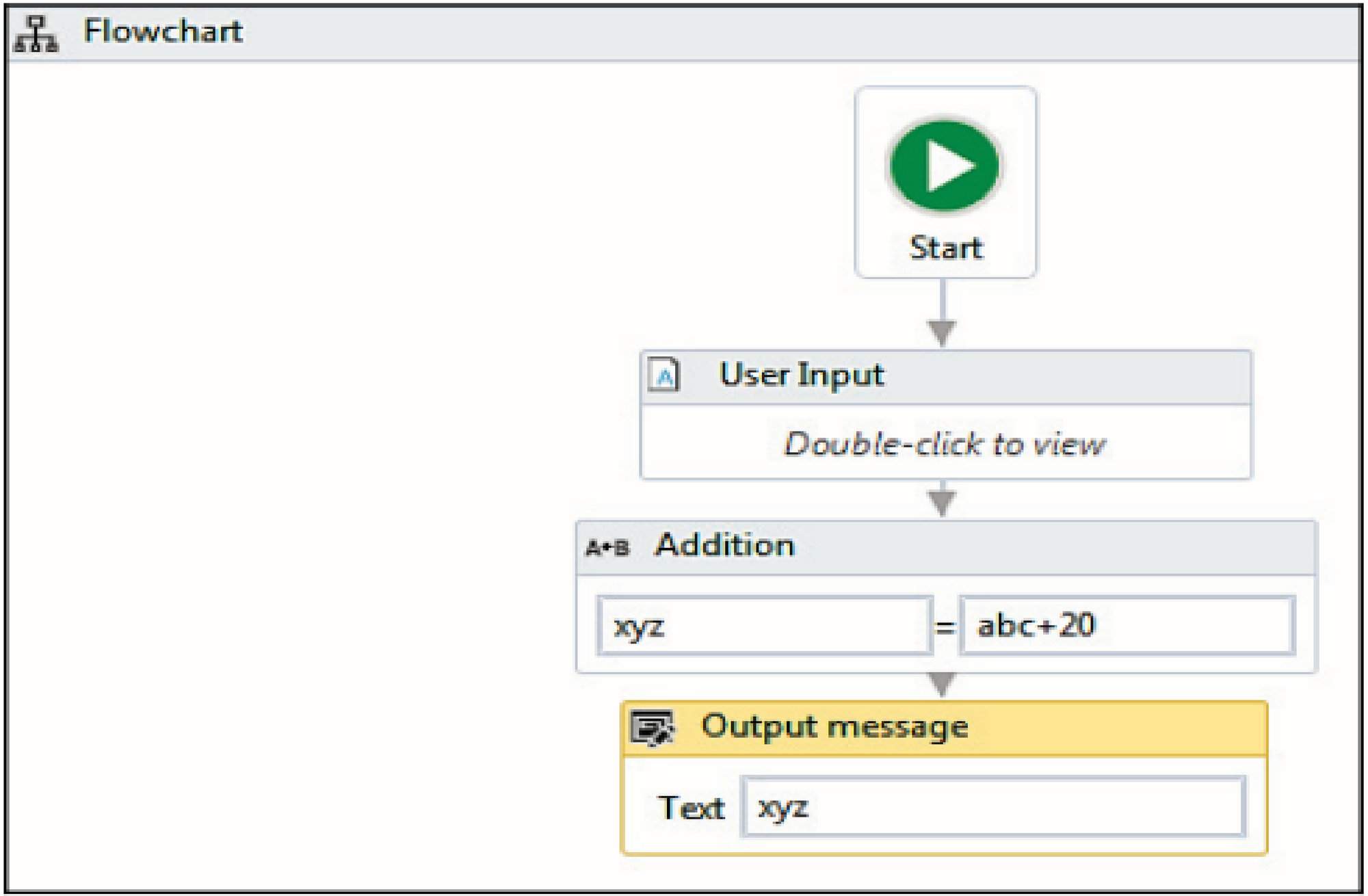
Using exception handling

- While working on a project, it is better to use exception handling because it reduces the risk of errors.
- For instance, using the Try catch block can give you a proper error message, which helps us handle exceptions.
- There are various exception handling techniques that have been explained earlier and that are very useful while working on a project.
- Here, we have used the Write line activity to display messages in the case of any error detected by the Catch block or the Finally block


Try catch
Type: TryCatch
Name: Try catch
Try
Write line
Text
Enter a VB expression
Catches
Exception
Add an activity
Add new catch
Finally
Add an activity

Making your workflow readable

- It is good practice to name activities on the basis of the operations they perform to ensure that when we return to the workflow, we can easily identify each and every step used in it.
- This becomes very helpful while finding and resolving errors as it specifies the process when showing an error during debugging.
- If activities are properly named, we get to know exactly which part of the workflow is not working.

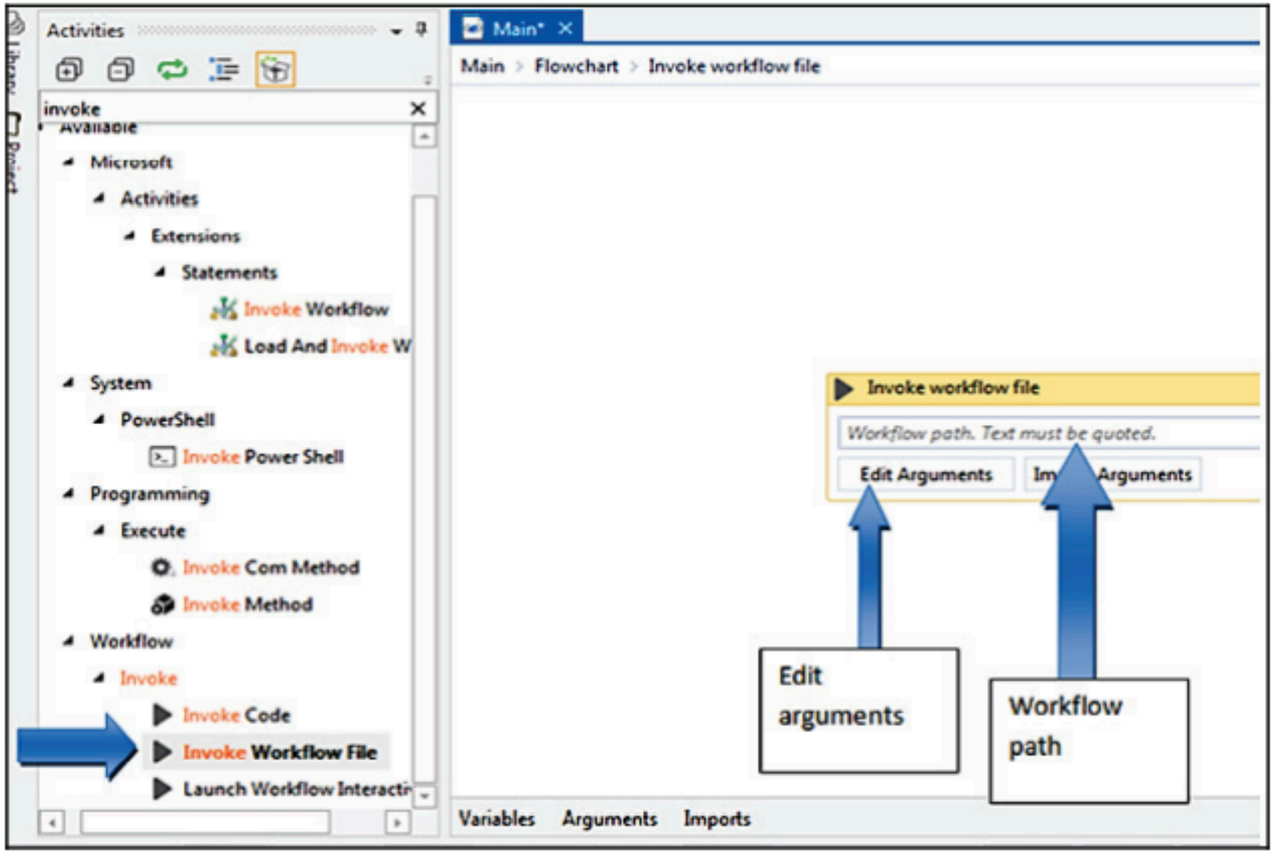

Flowchart
Start
User Input
Double-click to view
A+B Addition
xyz
=
abc+20
Output message
Text
xyz

- Keeping it clean Just as writing in a clean and understandable manner is the quality of a good coder, the same holds true for an RPA developer.
- Clean code helps us understand the whole process very easilydyou and whoever is reading it.

Nesting workflows

- While working in UiPath, it is better to divide the whole process into smaller parts and then nest these workflows into a larger one or the Main workflow.
- This can be done using the Invoke workflow file activity given in the Activities panel.
- There are several steps involved in nesting a workflow or many workflows into a single workflow

- How to nest a workflow inside a single workflow Say we have two workflows. In this example, we will invoke one workflow into the other:
- 1. Add an Invoke workflow file activity to the first workflow
- 2. Click on the Edit Arguments option available.
- 3. Define an argument and type it in the Invoke workflow arguments that appear
- 4. In the Arguments panel in the second workflow, create an argument with the same name as the first workflow. You will now be able to use that argument as any other variable

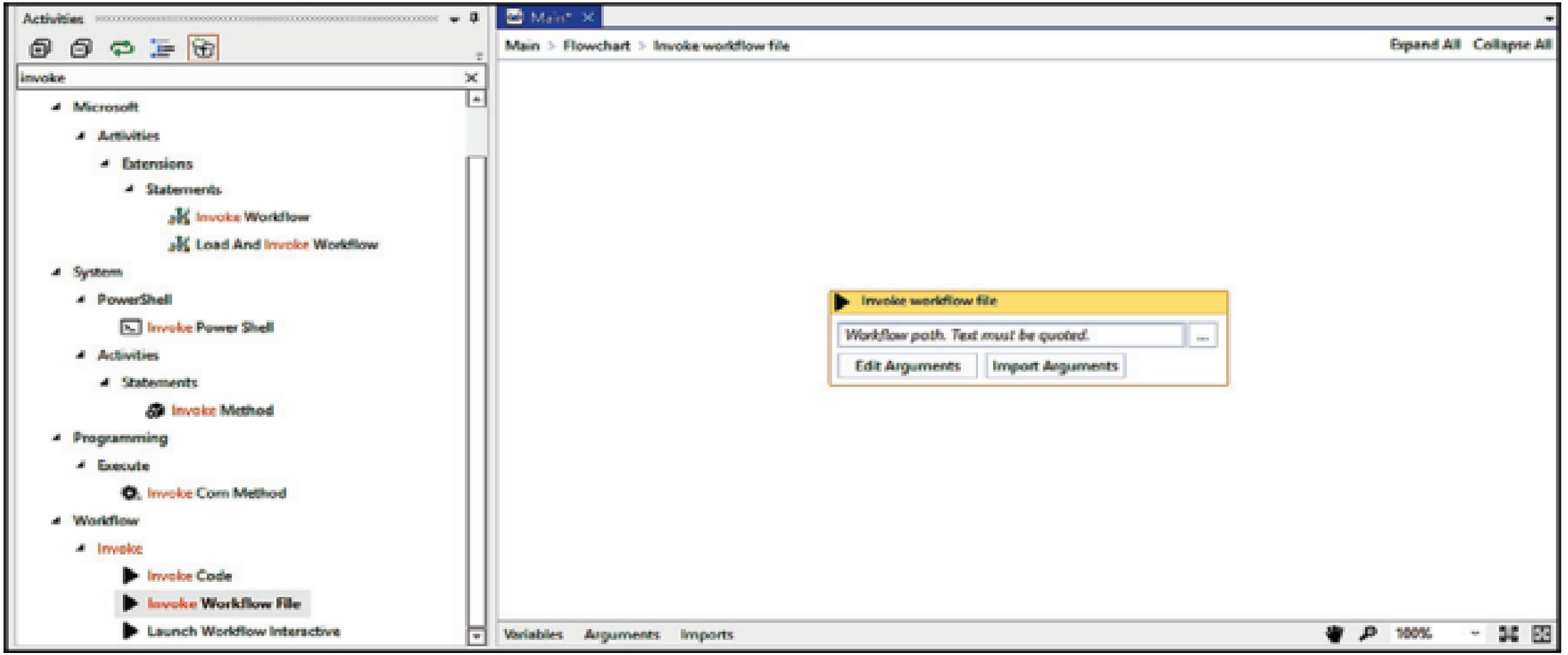

Activities
invoke
Microsoft
Activities
Extensions
Statements
Invoke Workflow
Load And Invoke W
System
PowerShell
Invoke Power Shell
Programming
Execute
Invoke Com Method
Invoke Method
Workflow
Invoke
Invoke Code
Invoke Workflow File
Launch Workflow Interacti
Main*
Main > Flowchart > Invoke workflow file
Invoke workflow file
Workflow path. Text must be quoted.
Edit Arguments
Arguments
Edit arguments
Workflow path
Variables
Arguments
Imports


SendResults
Main
SendResults
Expand All
Collapse All
Message box
Argument1
Name
Direction
Argument type
Default value
Argument1
In
String
Enter a VB expression

Reusability of workflows

- Reusing workflows makes the automation process easier and better since we can use earlier created workflows in our project that we are trying to use for automation.
- There are two methods for this:
  - Invoke workflow
  - file Templates

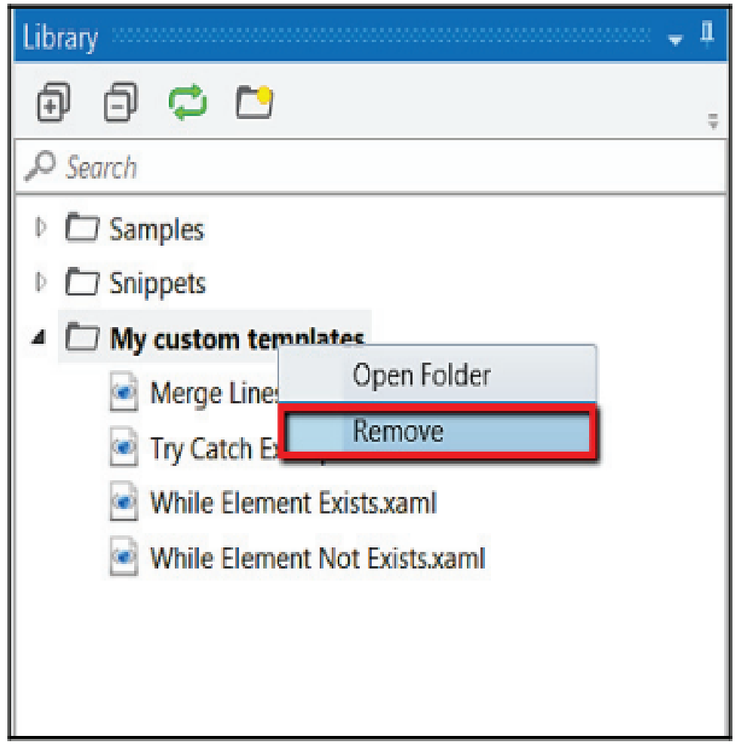
- Invoke workflow file is a good process if we have a complex automation project.
- We can divide it into smaller parts.
- By using the Invoke workflow file activity, we can invoke all those files in our project and collect all these smaller parts in a single workflow.
- However, if we want to invoke a previously created workflow in our project and make changes to the latter, the former will also get affected.
- Hence, it is recommended to use the Invoke workflow file activity only when we have a complex workflow and we want to divide the process into smaller parts and then use them together


Activities
invoke
Microsoft
Activities
Extensions
Statements
Invoke Workflow
Load And Invoke Workflow
System
PowerShell
Invoke Power Shell
Activities
Statements
Invoke Method
Programming
Execute
Invoke Com Method
Workflow
Invoke
Invoke Code
Invoke Workflow File
Launch Workflow Interactive
Main*
Main > Flowchart > Invoke workflow file
Expand All
Collapse All
Invoke workflow file
Workflow path. Text must be quoted.
...
Edit Arguments
Import Arguments
Variables
Arguments
Imports
100%

Templates

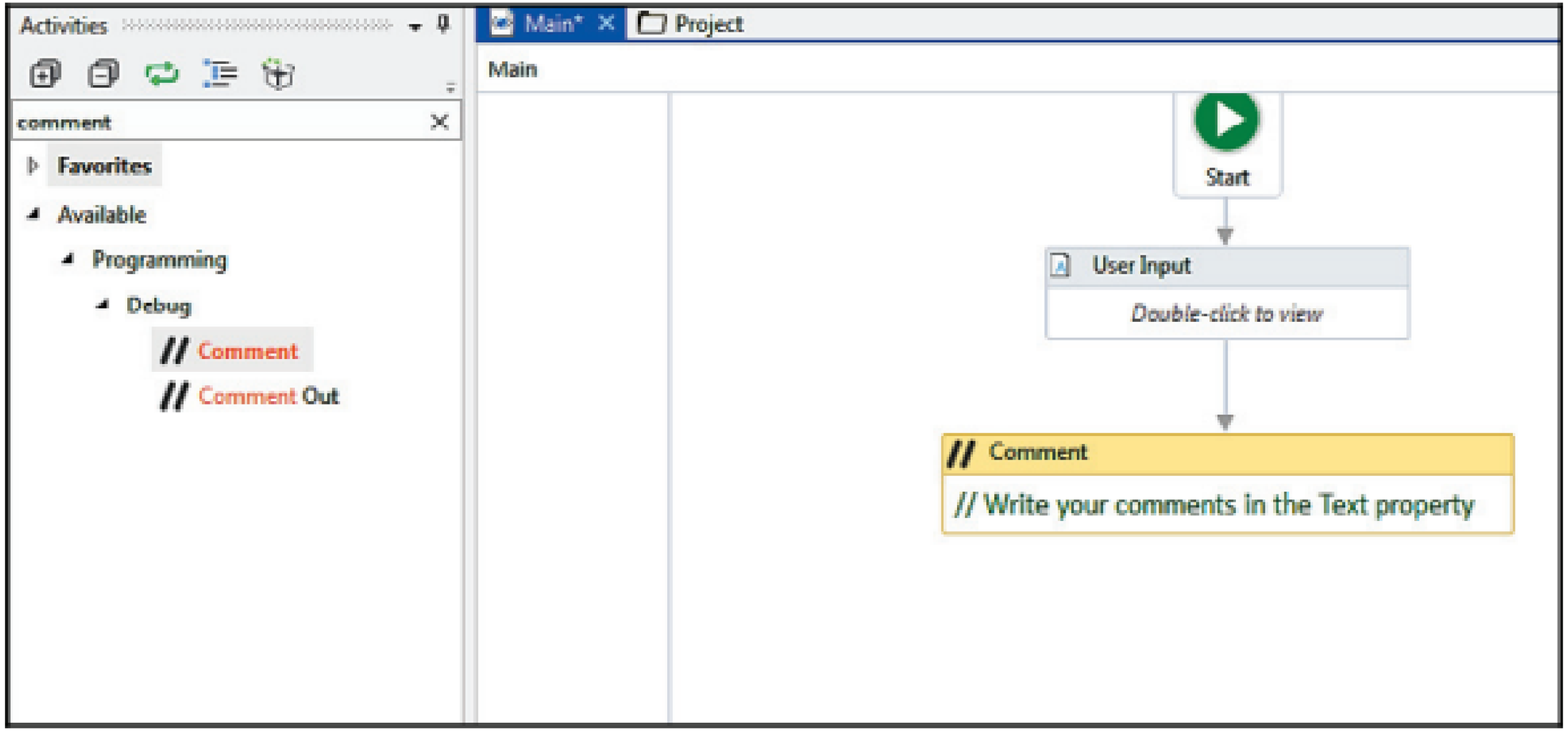
- Saving the workflow as a template helps you preserve the original workflow file.
- So whatever modifications you made in the template, no changes will be made in the original workflow.
- We often use templates when creating small pieces of common automation that are reusable and applicable in multiple workflows.
- So you can use templates if the workflow does not change over time.
- The most common example is when you create your own reusable snippets using data, data tables, and .xml files

- Adding a workflow as a template Follow the steps given to add a workflow as a template, which is explained as follows:
- 1. Add a new folder in the Library
- 2. Now browse for your file containing the workflows. Just select folder from the list that contains all the workflows
- 3. We can also remove an added file by just right-clicking on it and then selecting the Remove option



Commenting techniques

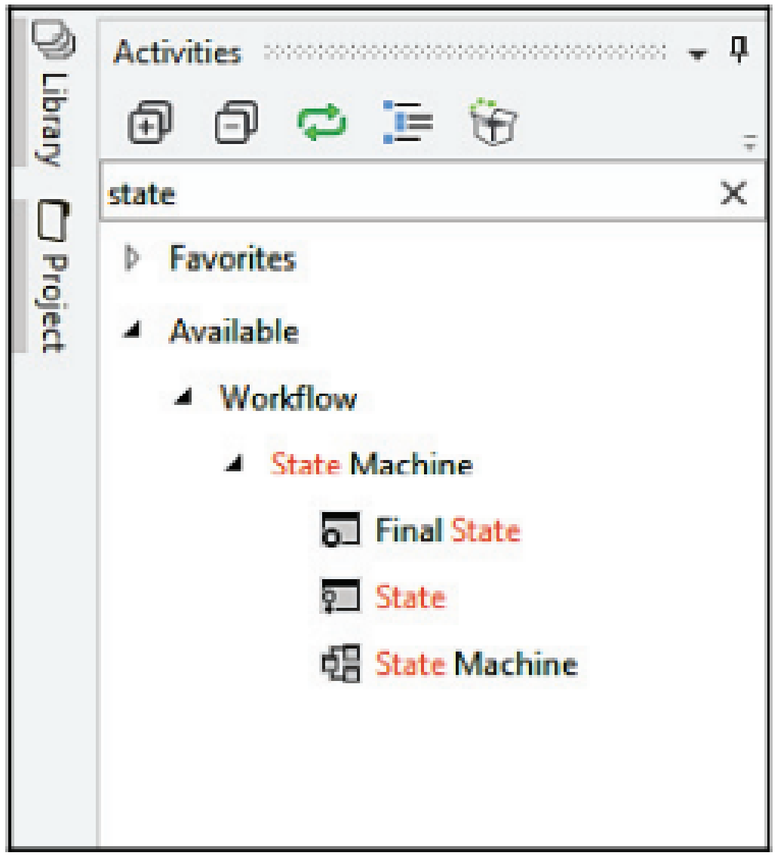
- Using comments in workflows is considered a good practice as it can give a better step-by-step notification of what is done in the workflow.
- Therefore, commenting in a complex workflow is considered to be good while debugging:
  - The package you'll need to use comments inside a workflow needs to be installed from the Package Manager functionality that is available in the Activities panel (the Manage Packages icon).
  - You can install UiPath.Core.Activities from the packages; inside you will find the Comment activity in the Activities panel as indicated by the arrow
  - Once the package is installed, just drag and drop the Comment activity from the Activities panel and add comments in between the workflows wherever you want

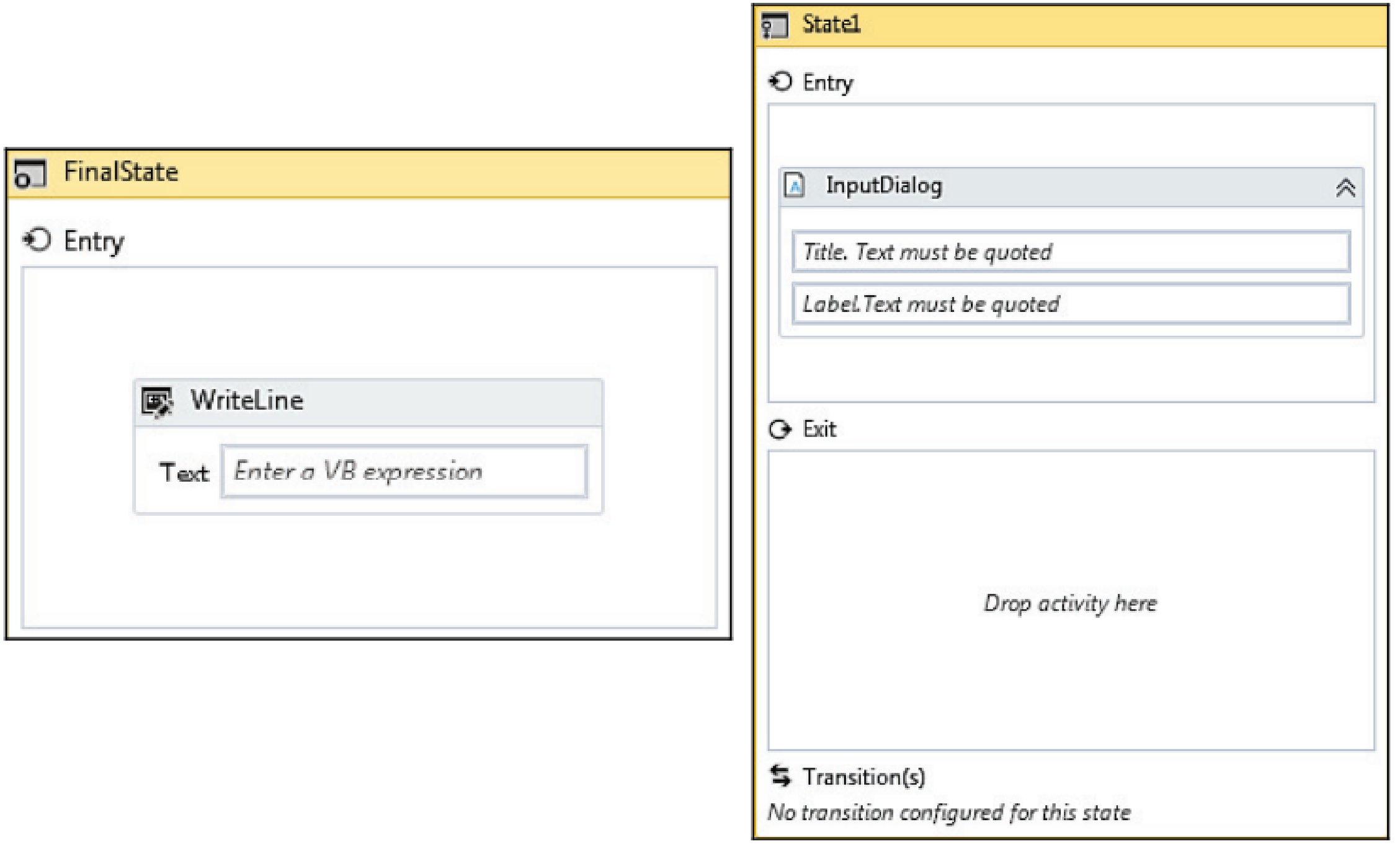

Activities
comment
Favorites
Available
Programming
Debug
Comment
Comment Out
Main*
Project
Main
Start
User Input
Double-click to view
Comment
// Write your comments in the Text property

State Machine

- A State Machine uses a finite number of sets in its execution. It can go into a state when it is triggered by an activity; it exits that state when another activity is triggered. Another important aspect of State Machines is transactions. They enable you to add conditions based on which transactions jump from one state to another. These are represented by arrows or branches between states
- There are two activities specific to State Machines. They are State and Final State

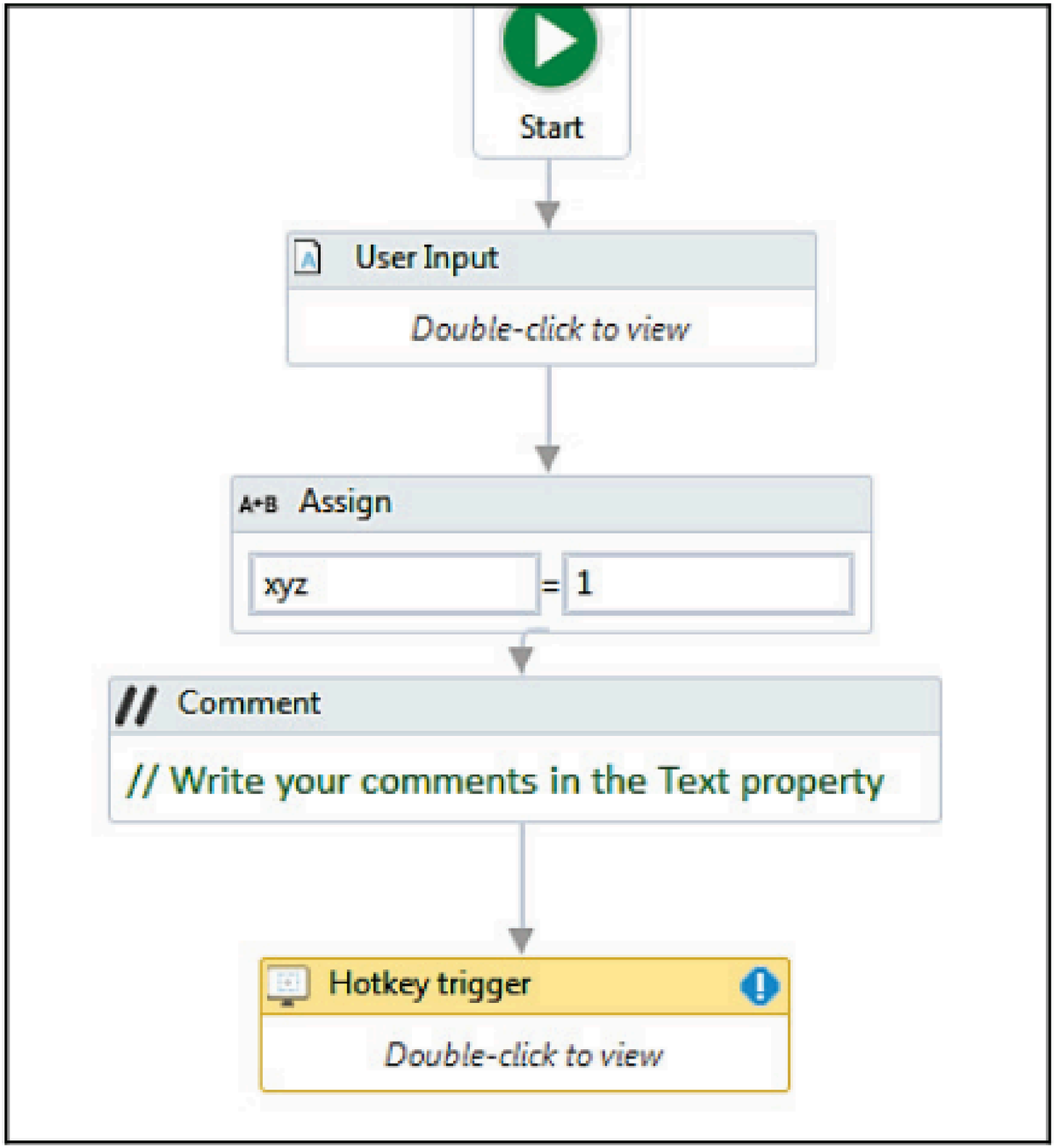
- The State activity consists of three sections
  - Entry, Exit, and Transitions, while the FinalState only contains Entry.
  - We can expand these activities by double-clicking them to view more information and to edit them
  - FinalState activity: This activity contains all those activities that need to be processed when the state is entered:
  - State activity: Transitions contain three sectionsdTrigger, Condition, and Action, which enable you to add a trigger for the next state or a condition under which an activity is to be executed:



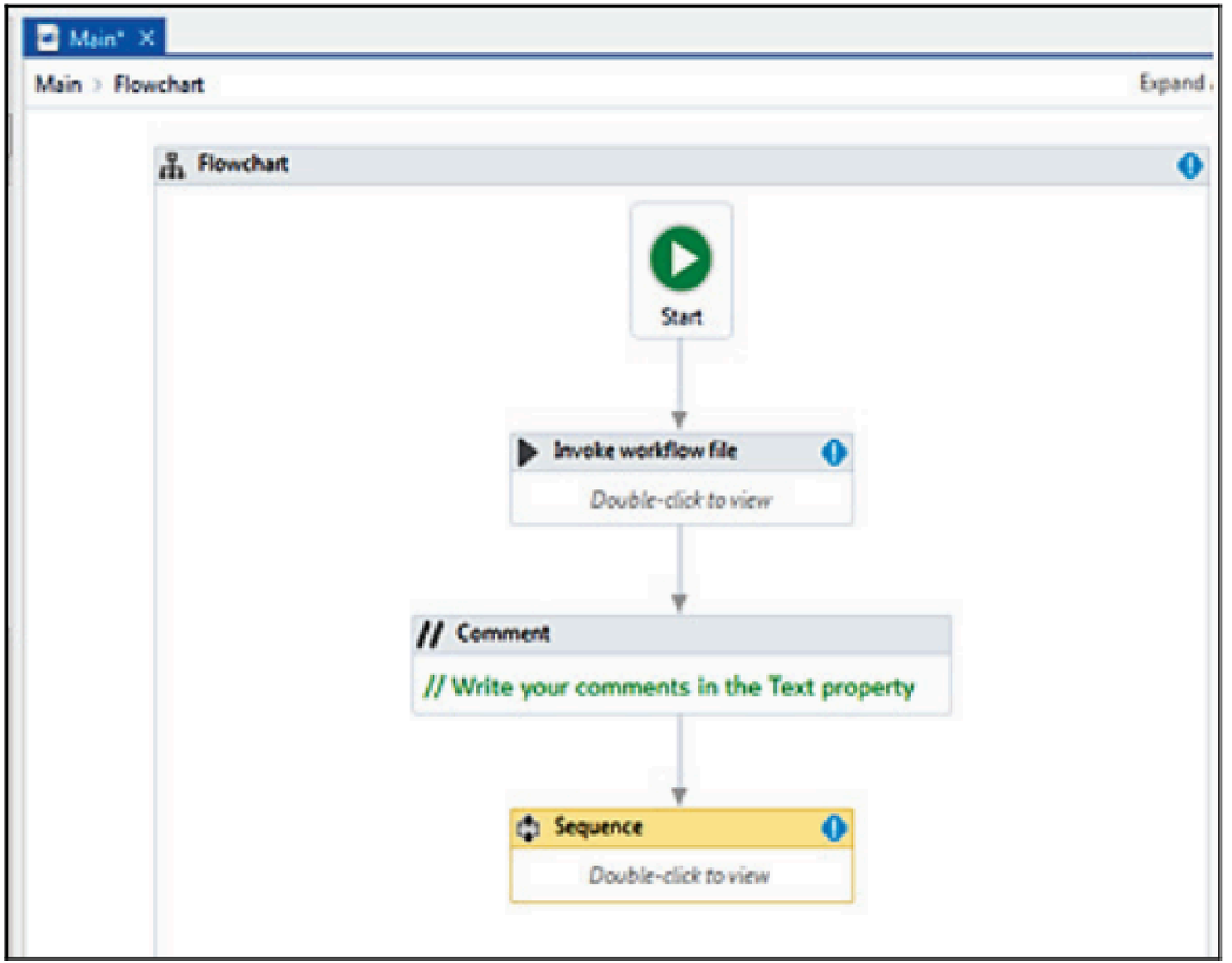

State1
Entry
InputDialog
Title. Text must be quoted
Label.Text must be quoted
Exit
Drop activity here
Transition(s)
No transition configured for this state
FinalState
Entry
WriteLine
Text
Enter a VB expression

When to use Flowcharts, State Machines, or Sequences

- A Sequence is used only when we have a selected a straightforward set of instructions on how to create a workflow.
- That is, we do not have to make decisions.
- It is preferred when we are recording some steps in a sequential manner and we are creating a simple workflow

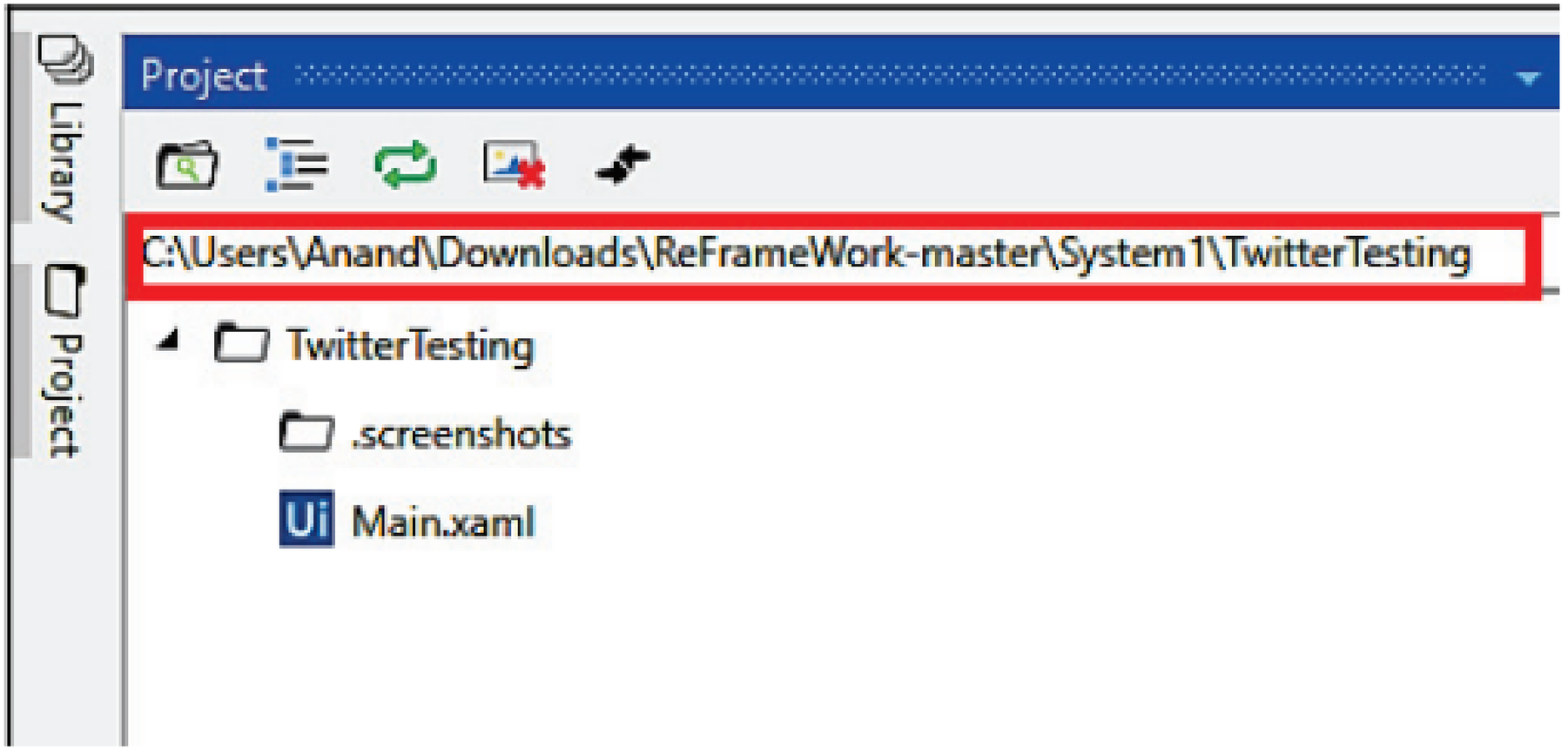

Start
User Input
Double-click to view
A+B Assign
xyz
= 1
Comment
// Write your comments in the Text property
Hotkey trigger
Double-click to view

- Now, when it comes to State Machines and Flowcharts, both are used for complex processes and both work well.
- They work in the same manner, but State Machines have some advantages over Flowcharts, which are listed as follows:
  - Complex transitions are much clearer with State Machines as they have an inbuilt layout of the workflow.
  - Flowcharts do not inherently have the concept of waiting for something to happen.
    - State Machines do (a transition will not occur until a trigger completes and the condition evaluates to true). State Machines naturally encapsulate the action group


Main*
Main > Flowchart
Expand
Flowchart
Start
Invoke workflow file
Double-click to view
Comment
// Write your comments in the Text property
Sequence
Double-click to view

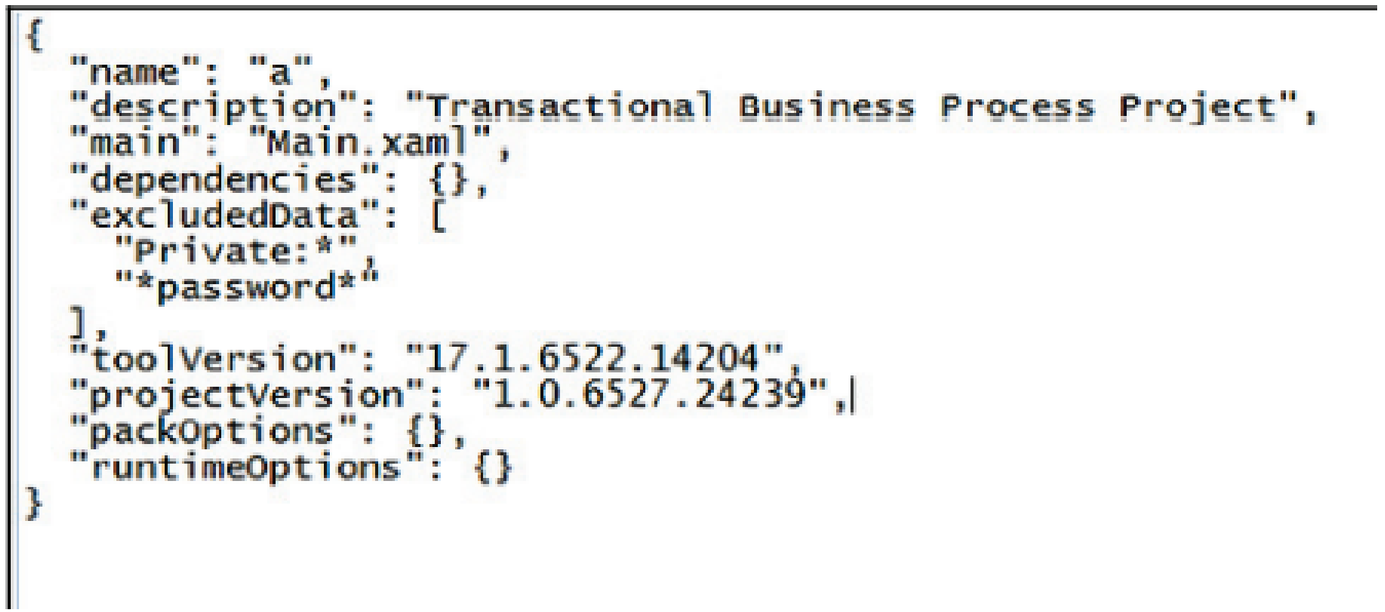
Using config files and examples of a config file

- When it comes to configuration,
  - UiPath does not have any pre-built configuration file such as Visual Studio, but we can create one.
  - It is considered to be one of the best practices to keep environment settings in a config file so that they can be easily changed by the user when required.
- Thus, when we create a project, the Project.json file that holds all the activities is created automatically.
- Project.json can be found in the folder where the project is saved. To access the folder, we can just open the Project, then copy the path (as shown in the following screenshot), and paste it into File Explorer
- Then you can see a project.json file in File Explorer like the one shown in the following screenshot



Name	Date modified	Type	Size
Initialization	11/14/2017 1:28 PM	XAML File	5 KB
Main	11/14/2017 1:28 PM	XAML File	36 KB
project	11/14/2017 1:28 PM	JSON File	1 KB
Transaction	11/14/2017 1:28 PM	XAML File	9 KB

- The following screenshot displays the code inside the project.json file, when you open that file in Notepad



```
{
  "name": "a",
  "description": "Transactional Business Process Project",
  "main": "Main.xaml",
  "dependencies": {},
  "excludedData": [
    "Private:*"
    "*password*"
  ],
  "toolVersion": "17.1.6522.14204",
  "projectVersion": "1.0.6527.24239",
  "packOptions": {},
  "runtimeOptions": {}
}
```

- You can also store your settings with the help of a spreadsheet or credentials. There are various parameters contained in the “Project.json” file.
  - Name: This is the title of the project that is provided when creating a project in the create New Project window
  - Description: When creating a project, a description is also required. You can add the description in the Create New Project window
  - Main: This is the entry point for the project. It is saved as “main.xml”by default, but you can change its name from the Project panel.
    - Also, you have multiple workflows for a project, it is necessary to attach all these files to the main file with the Invoke Workflow File activity.
    - Otherwise, those files will not be executed

- Dependencies: These are the activities packages that are used in a project and their versions.
- Excluded data: Contains keyword that can be added to the name of an activity to prevent the variable and argument values from being logged at the verbose level.
- Tool version: The version of Studio used to create a project

- Adding Credential: We can add particular settings that can be used further. For example, we can save the username and password to be used further, so this can be done with the help of the Add credential activity that can be found in the Activities panel
- After adding credentials, type the required values in the Properties panel
- So, when the credentials are set, we can delete, secure, or request credentials, as shown in the following steps:
- 1. Delete Credentials: If we want to delete a credential then we can just drag and drop the Delete Credentials activity and then define the target for the credential

- 2. Get Secure Credential: This is used to get the values, that is, the username and password, that were set during the addition of a credential. We have to set the target the same as before; the output will be the username and password
- 3. Request Credential: This is a property in which the robot displays a message dialog asking the user for username and password and stores this information as a string. This can then be used in further processes. The user can select OK to provide credentials or even cancel it if they do not want to provide credentials